Telegram to Notesnook Bridge: Deployment Guide (v1.1)

Oct 15, 2026 · Robb Joseph

What you'll build

You'll run a small private Telegram bot that saves every message you send it as a note in your Notesnook account, formatting, checklists, photos and text files included. The bot runs on Fly.io, so it keeps working when your own computer is off.

1. You send a message to your bot in Telegram.
2. The bot, running on Fly.io, converts the message to HTML.
3. It posts the note to Notesnook's Inbox API using your API key.
4. The note appears in Notesnook after your next sync, on every device.

The whole setup takes about 30 minutes. You need:

- A **Telegram** account.
- A **Notesnook** account with the Inbox API available. It was added in Notesnook 3.4 and is still labeled beta, so check Settings > Inbox in your app.
- A **Fly.io** account. Fly asks for a payment card, and the bot runs on its smallest machine. Check Fly.io pricing for current rates.
- A computer with a terminal. You don't need to install Python locally; Fly builds the bot for you.

The bot only answers you. Anyone else who finds it is ignored.

Step 1: Create the Telegram bot

You need two values from Telegram: a bot token and your own numeric user ID.

1. In Telegram, open a chat with **@BotFather**.
2. Send `/newbot`. Pick a display name, then a username that ends in `bot`.
3. BotFather replies with a token that looks like `123456789:ABCdef...`. Copy it. Anyone with this token controls your bot, so treat it like a password.
4. Open a chat with **@userinfobot** and copy the number it replies with. This is your user ID. The bot will only respond to this ID.

Keep both values handy. You'll store them as secrets in Step 4.

Step 2: Turn on the Notesnook Inbox API

The Inbox API is a private mailbox for your account. Other services post notes to it with a key, and the notes show up after your next sync, encrypted so only you can read them.

1. In Notesnook, open **Settings > Account > Inbox**. On some versions it is **Settings > Inbox**.
2. Turn on **Enable Inbox API**. Unless you have your own PGP key pair, let Notesnook generate the keys.
3. Under API Keys, click **Create Key**. Name it something like `telegram` and choose an expiry.
4. Copy the key and keep it private.

The default expiry is one month. When a key expires, the bot starts replying with a 401 error. Choose a longer expiry or Never if you don't want to rotate it, and see the troubleshooting section for how to replace a key.

The Inbox API is still in beta, so details may change. The official guide has the current request format.

Step 3: Set up the project files

Create a new empty folder, for example `notesnook-bot`, and save these five files in it. Use a dedicated folder: Fly uploads everything in the folder you deploy from, so a crowded folder makes deploys slow.

bot.py

```
"""Telegram -> Notesnook Inbox API bridge.

Send a message to your Telegram bot and it shows up as a note in Notesnook
after your next sync. Bold, italic, links, code and quotes are kept.

The Inbox API has no attachment field, so media is embedded in the note's
HTML, the way the Notesnook email worker does it: photos and image files
become base64 data: URIs and text files become code blocks. Videos, voice
notes and other files get a placeholder line (plus the caption, if any).
```

```
Env vars:
  TELEGRAM_BOT_TOKEN   token from @BotFather
  NOTESNOOK_INBOX_KEY  key from Notesnook > Settings > Account > Inbox
  ALLOWED_USER_ID      your numeric Telegram user ID (only you can use the
bot)
  BOT_TIMEZONE         optional, default America/New_York (generic note
titles)
  EMBED_MEDIA          optional, set to 0 to turn off embedding (placeholders
only)
"""

import asyncio
import base64
import html
import os
import re
from datetime import timezone
from zoneinfo import ZoneInfo

import requests
from telegram import BotCommand, Message, Update
from telegram.ext import (
    Application,
    CommandHandler,
    ContextTypes,
    MessageHandler,
    filters,
)

INBOX_URL = "https://inbox.notesnook.com/"
BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
INBOX_KEY = os.environ["NOTESNOOK_INBOX_KEY"]
ALLOWED_USER_ID = int(os.environ["ALLOWED_USER_ID"])
TIMEZONE = os.environ.get("BOT_TIMEZONE", "America/New_York")  # for generic
titles
__version__ = "1.1.0"  # bump when you change the bot; shown by /ver

ATTACHMENT_SIZE_LIMIT = 5 * 1024 * 1024  # per file, same as the email worker
NOTE_SIZE_LIMIT = 8_000_000  # bytes of HTML per note (the Inbox API allows
10 MB)
EMBED_MEDIA = os.environ.get("EMBED_MEDIA", "1").strip().lower() not in (
    "0",
    "false",
```

```
    "no",
    "off",
)

_IMAGE_MIMES = {"image/jpeg", "image/png", "image/gif", "image/webp"}
_TEXT_MIMES = {
    "application/json",
    "application/xml",
    "application/x-yaml",
    "application/yaml",
    "application/toml",
    "application/javascript",
    "application/x-sh",
}
_TEXT_EXTS = {
    ".txt", ".md", ".markdown", ".csv", ".tsv", ".json", ".xml", ".yaml",
".yml",
    ".toml", ".ini", ".log", ".py", ".js", ".ts", ".html", ".css", ".sh",
".sql",
}

# Telegram-only tags Notesnook doesn't know; drop the tag, keep the text.
_TG_ONLY_TAGS = re.compile(r"</?tg-(?:spoiler|emoji)[^>]*>")
_PRE_BLOCK = re.compile(r"(<pre>.*?</pre>)", re.DOTALL)
_BLOCK_TAGS = ("<pre>", "<blockquote")


def media_label(msg: Message) -> str | None:
    """Return a human label if the message carries media, else None."""
    if msg.photo:
        return "Photo"
    for attr, label in (
        ("video", "Video"),
        ("video_note", "Video message"),
        ("animation", "GIF"),
        ("voice", "Voice note"),
        ("audio", "Audio"),
        ("document", "File"),
        ("sticker", "Sticker"),
    ):
        if getattr(msg, attr, None):
            return label
    return None
```

```
_TAG = re.compile(r"<(/?)([a-z][a-z0-9-]*)[^>]*>")
_TASK = re.compile(r"^[-*•]\s+\[([ xX]?)\]\s+(.+)$")
_BULLET = re.compile(r"^[-*•]\s+(.+)$")
_ORDERED = re.compile(r"^\d+[.)]\s+(.+)$")
_HEADING = re.compile(r"^(#{1,6})\s+(.+)$")
_LIST_START = re.compile(r"^(?:[-*•]\s+|\d+[.)]\s+)\S")
_HEADING_MARK = re.compile(r"^#{1,6}\s+")


def _balanced(fragment: str) -> bool:
    """True if every tag opened in the fragment is closed inside it."""
    stack = []
    for m in _TAG.finditer(fragment):
        closing, name = m.group(1), m.group(2)
        if closing:
            if not stack or stack.pop() != name:
                return False
        else:
            stack.append(name)
    return not stack


def _plain_title(line_html: str) -> str:
    """Plain-text version of a line of Telegram HTML, minus a trailing
colon."""
    return html.unescape(_TAG.sub("", line_html)).strip().rstrip(":").strip()


def _title_attr(line_html: str | None) -> str:
    """data-title attribute: Notesnook shows it as a checklist's header
title."""
    plain = _plain_title(line_html) if line_html else ""
    return f' data-title="{html.escape(plain[:80], quote=True)}"' if plain
else ""


def _structured_html(tg_html: str) -> str | None:
    """Convert markdown-style lines (lists, todos, headings) to real HTML.

    Returns None when the message has no such lines, or when it isn't safe to
    split it into lines (formatting that spans lines, code blocks, quotes);
```

```
    the caller then falls back to the simple <br> conversion.
    """
    if "<pre" in tg_html or "<blockquote" in tg_html:
        return None
    lines = tg_html.split("\n")
    if not all(_balanced(line) for line in lines):
        return None

    parsed = []  # (kind, text, extra)
    for line in lines:
        s = line.strip()
        if not s:
            parsed.append(("blank", "", None))
        elif m := _TASK.match(s):
            parsed.append(("task", m.group(2), m.group(1).lower() == "x"))
        elif m := _BULLET.match(s):
            parsed.append(("bullet", m.group(1), None))
        elif m := _ORDERED.match(s):
            parsed.append(("ordered", m.group(1), None))
        elif m := _HEADING.match(s):
            parsed.append(("heading", m.group(2), len(m.group(1))))
        else:
            parsed.append(("text", s, None))

    if not any(k in ("task", "bullet", "ordered", "heading") for k, _, _ in
parsed):
        return None

    def next_is_task(idx: int) -> bool:
        return idx < len(parsed) and parsed[idx][0] == "task"

    out = []
    pending_title = None  # line directly above a checklist -> its header
title
    i = 0
    while i < len(parsed):
        kind, text, extra = parsed[i]
        if kind == "blank":
            i += 1
        elif kind == "heading":
            if next_is_task(i + 1) and _plain_title(text):
                pending_title = text
            else:
```

```
                out.append(f"<h{extra}>{text}</h{extra}>")
            i += 1
        elif kind in ("task", "bullet", "ordered"):
            items = []
            while i < len(parsed) and parsed[i][0] == kind:
                _, t, checked = parsed[i]
                items.append(f'<li class="checked">{t}</li>' if checked else
f"<li>{t}</li>")
                i += 1
            opener = {
                "task": '<ul class="checklist"' + _title_attr(pending_title)
+ ">",
                "bullet": "<ul>",
                "ordered": "<ol>",
            }[kind]
            closer = "</ol>" if kind == "ordered" else "</ul>"
            out.append(opener + "".join(items) + closer)
            pending_title = None
        else:  # run of plain text lines -> one paragraph
            run = []
            while i < len(parsed) and parsed[i][0] == "text":
                run.append(parsed[i][1])
                i += 1
            if next_is_task(i) and _plain_title(run[-1]):
                pending_title = run.pop()
            if run:
                out.append("<p>" + "<br>".join(run) + "</p>")
    return "".join(out)


def telegram_html_to_note_html(tg_html: str) -> str:
    """Turn Telegram's HTML (tags already balanced and escaped) into note
HTML.

    Markdown-style lists, todos and headings become real HTML lists when it
is
    safe to (see _structured_html). Otherwise newlines become <br> outside
    <pre> blocks. We deliberately avoid splitting into multiple <p> tags in
    that case: a bold span can cross a blank line, and Notesnook turns the
    whole note into a code block if any tags are unbalanced.
    """
    tg_html = _TG_ONLY_TAGS.sub("", tg_html.strip())
    structured = _structured_html(tg_html)
```

```
    if structured is not None:
        return structured
    parts = _PRE_BLOCK.split(tg_html)
    out = [
        part if part.startswith("<pre>") else part.replace("\n", "<br>")
        for part in parts
    ]
    body = "".join(out)
    if any(tag in body for tag in _BLOCK_TAGS):
        return body  # block elements can't live inside <p>
    return f"<p>{body}</p>"


def generic_title(msg: Message) -> str:
    """e.g. 'Telegram note, Oct 15, 8:15 AM' in the BOT_TIMEZONE zone."""
    dt = msg.date
    if dt.tzinfo is None:
        dt = dt.replace(tzinfo=timezone.utc)
    d = dt.astimezone(ZoneInfo(TIMEZONE))
    return f"Telegram note, {d:%b} {d.day}, {d.hour % 12 or 12}:
{d.minute:02d} {d:%p}"


async def embed_attachment(msg: Message) -> tuple[str | None, str | None]:
    """Download a photo, image file or text file and return (html, None).

    Mirrors the Notesnook email worker: images become base64 data: URIs, text
    files become code blocks. Anything else, or anything over
    ATTACHMENT_SIZE_LIMIT, returns (None, reason) and gets a placeholder.
    """
    if not EMBED_MEDIA:
        return None, "embedding is turned off"

    name = None
    if msg.photo:
        tg_file, mime, kind = msg.photo[-1], "image/jpeg", "image"
    elif msg.document:
        tg_file = msg.document
        mime = (tg_file.mime_type or "").lower()
        name = tg_file.file_name or "file"
        ext = os.path.splitext(name)[1].lower()
        if mime in _IMAGE_MIMES:
            kind = "image"
```

```
        elif mime.startswith("text/") or mime in _TEXT_MIMES or ext in
_TEXT_EXTS:
            kind = "text"
        else:
            return None, "only images and text files can be embedded"
    else:
        return None, "only photos, images and text files can be embedded"

    limit_mb = ATTACHMENT_SIZE_LIMIT // (1024 * 1024)
    if tg_file.file_size and tg_file.file_size > ATTACHMENT_SIZE_LIMIT:
        return None, f"too big ({tg_file.file_size / 1048576:.1f} MB, limit
{limit_mb} MB)"
    try:
        downloaded = await tg_file.get_file()
        data = bytes(await downloaded.download_as_bytearray())
    except Exception as e:  # Telegram API or network error
        return None, f"download failed ({type(e).__name__})"
    if len(data) > ATTACHMENT_SIZE_LIMIT:
        return None, f"too big (limit {limit_mb} MB)"

    heading = f"<h3>{html.escape(name)}</h3>" if name else ""
    if kind == "image":
        b64 = base64.b64encode(data).decode("ascii")
        return f'{heading}<p><img src="data:{mime};base64,{b64}" /></p>',
None
    if b"\x00" in data:
        return None, "file looks binary, not text"
    text = data.decode("utf-8", errors="replace")
    return f"{heading}<pre><code>{html.escape(text, quote=False)}</code>
</pre>", None


def build_note(
    msg: Message, attachment_html: str | None = None, problem: str | None =
None
) -> tuple[str, str] | None:
    """Return (title, html) for a message, or None if there's nothing to
save.

    attachment_html is an embedded photo/file block from embed_attachment;
when
    a message has media but none is given, a placeholder line is added
instead
```

```
    (with `problem` as the reason it wasn't embedded).
    """
    label = media_label(msg)
    plain = (msg.text or msg.caption or "").strip()
    rich = (msg.text_html if msg.text else msg.caption_html) or ""

    if not plain and not label:
        return None

    body_parts = []
    if plain:
        body_parts.append(telegram_html_to_note_html(rich))
    if label:
        if attachment_html:
            body_parts.append(attachment_html)
        else:
            why = f", not embedded: {html.escape(problem)}" if problem else
""
            body_parts.append(f"<p><i>[{label} attached in Telegram{why}]</i>
</p>")

    if plain:
        first_line = plain.splitlines()[0].strip()
        if _LIST_START.match(first_line):
            # Starts with a list item: a generic title avoids duplicating it.
            title = generic_title(msg)
        else:
            first_line = _HEADING_MARK.sub("", first_line) or first_line
            first_line = first_line.rstrip(":").strip() or first_line
            title = (first_line[:80] + "…") if len(first_line) > 80 else
first_line
    else:
        file_name = msg.document.file_name if msg.document else None
        title = file_name or f"{label} from Telegram"

    return title, "".join(body_parts)


def send_to_notesnook(title: str, content_html: str) -> requests.Response:
    payload = {
        "title": title,
        "type": "note",
        "source": "telegram",
```

```
        "version": 1,
        "content": {"type": "html", "data": content_html},
    }
    return requests.post(
        INBOX_URL,
        json=payload,
        headers={"Authorization": INBOX_KEY},
        timeout=20,
    )


async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = update.effective_message
    if msg is None or update.effective_user.id != ALLOWED_USER_ID:
        return  # ignore everyone else

    attachment_html, problem = None, None
    if media_label(msg):
        attachment_html, problem = await embed_attachment(msg)

    note = build_note(msg, attachment_html, problem)
    if note is None:
        await msg.reply_text("Nothing to save in that message.")
        return
    title, content_html = note

    if attachment_html and len(content_html.encode("utf-8")) >
NOTE_SIZE_LIMIT:
        attachment_html, problem = None, "the note would be too large"
        title, content_html = build_note(msg, None, problem)

    try:
        resp = await asyncio.to_thread(send_to_notesnook, title,
content_html)
    except requests.RequestException as e:
        await msg.reply_text(f"Network error: {e}")
        return

    if resp.status_code == 200:
        suffix = ""
        if media_label(msg):
            suffix = " (media embedded)" if attachment_html else f" (media
not embedded: {problem})"
```

```
        await msg.reply_text(f"✓ Saved to Notesnook{suffix}")
    else:
        await msg.reply_text(f"Failed ({resp.status_code}):
{resp.text[:300]}")


HELP_TEXT = """Telegram -> Notesnook bridge

Send this bot a message to save it as a note in Notesnook. Notes appear in
Notesnook after its next sync.

COMMANDS
/help  show this guide
/ver   show the bot version

LISTS, TODOS AND HEADINGS
Start a line with one of these:
- item (or * item, • item)  ->  bullet list
1. item (or 1) item)  ->  numbered list
- [ ] item  ->  todo
- [x] item  ->  completed todo
# Heading (up to ###### Heading)  ->  heading

Example:
Groceries:
- [ ] milk
- [x] eggs

A line directly above a todo list, with no blank line between, becomes the
title of that checklist. In the example, "Groceries" is the checklist title.

NOTE TITLES
The first line of a message is the note title. A message that starts with a
list item gets a title with the date and time instead, like "Telegram note,
Oct 15, 8:15 AM".

FORMATTING
Select text and use Telegram's formatting options to apply bold, italic,
underline, strikethrough, links, monospace (code) and quotes. All of these
carry over to the note. Spoiler text is saved as plain text.

HTML
Tags typed by hand, such as <b>, are not interpreted and appear in the note
```

```
as plain text. Use Telegram's formatting options instead.

PHOTOS AND FILES
• Photos and image files (JPEG, PNG, GIF, WebP) are embedded in the note.
• Text files (.txt, .md, .csv, .json, code and similar) are added as code
blocks.
• Files can be up to 5 MB each.
• Videos, voice notes and other file types are not embedded. The note shows a
placeholder line, plus the caption if there is one.
• Embedded images are part of the note itself, not Notesnook attachments. To
save a real attachment, share the file to Notesnook from your phone's share
sheet.
• Each photo in an album becomes its own note.

LIMITS
• Lists can't be nested, and numbering always starts at 1.
• A message that contains a code block, a quote, or bold text spanning
several lines is saved as simple text, without list conversion."""


def _authorized(update: Update) -> bool:
    user = update.effective_user
    return user is not None and user.id == ALLOWED_USER_ID


async def cmd_help(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if _authorized(update):
        await update.effective_message.reply_text(HELP_TEXT)


async def cmd_ver(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not _authorized(update):
        return
    lines = [f"Telegram -> Notesnook bridge v{__version__}"]
    if region := os.environ.get("FLY_REGION"):
        lines.append(f"Running on Fly.io (region: {region})")
    lines.append(f"Time zone for generic titles: {TIMEZONE}")
    lines.append(f"Media embedding: {'on' if EMBED_MEDIA else 'off'}")
    await update.effective_message.reply_text("\n".join(lines))


async def cmd_unknown(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if _authorized(update):
```

```
        await update.effective_message.reply_text("Unknown command. Send
/help.")


async def post_init(app: Application):
    await app.bot.set_my_commands(
        [
            BotCommand("help", "How to use this bot"),
            BotCommand("ver", "Show the bot version"),
        ]
    )


def main():
    app = (
        Application.builder()
        .token(BOT_TOKEN)
        .read_timeout(60)  # file downloads can be slow
        .write_timeout(60)
        .post_init(post_init)
        .build()
    )
    app.add_handler(CommandHandler(["start", "help"], cmd_help))
    app.add_handler(CommandHandler("ver", cmd_ver))
    app.add_handler(MessageHandler(filters.COMMAND, cmd_unknown))
    app.add_handler(MessageHandler(~filters.COMMAND, handle_message))
    app.run_polling()


if __name__ == "__main__":
    main()
```

requirements.txt

```
python-telegram-bot>=21.0
requests>=2.31
tzdata
```

Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY bot.py .

CMD ["python", "bot.py"]
```

fly.toml

Change `app` to a name of your own; it must be unique across all of Fly.io. `iad` is Virginia, so pick a region near you if you like (`fly platform regions` lists them).

```
# Change this to a globally unique name, e.g. "yourname-notesnook-bot"
app = "your-unique-app-name"
primary_region = "iad"

[build]

# No [http_service]: the bot polls Telegram, so it needs no public port.

[[vm]]
  size = "shared-cpu-1x"
  memory = "256mb"
```

.dockerignore

```
# Ignore everything, then allow only what the image needs.
*
!bot.py
!requirements.txt
```

The file name starts with a dot. On a Mac, press Cmd+Shift+. in Finder to show hidden files, and check that it wasn't saved as `dockerignore`. It stops Fly from uploading anything else in the folder.

Step 4: Deploy to Fly.io

Run every command below from inside your project folder.

1. **Install the Fly CLI.**
   - Windows (PowerShell): `iwr https://fly.io/install.ps1 -useb | iex`
   - Mac or Linux: `curl -L https://fly.io/install.sh | sh`
2. **Sign up or log in.** Run `fly auth signup` for a new account, or `fly auth login` if you already have one.
3. **Create the app.** Use the same name you put in `fly.toml`:

   ```
   fly apps create your-unique-app-name
   ```

4. **Store your secrets.** They stay encrypted on Fly and never appear in your code. Use the three values from Steps 1 and 2:

   ```
   fly secrets set TELEGRAM_BOT_TOKEN="your-bot-token"
   NOTESNOOK_INBOX_KEY="your-inbox-key" ALLOWED_USER_ID="123456789"
   ```

   Optional: the bot titles list notes with the time, in New York time by default. To change the zone, add `BOT_TIMEZONE="Europe/London"` or any other tz database name.
   Optional: to stop the bot embedding photos and files, and save a placeholder line instead, add `EMBED_MEDIA="0"` the same way.
5. **Deploy.**

   ```
   fly deploy
   ```

6. **Keep it to one machine.** Fly may start two by default, and two copies polling Telegram will fight each other.

   ```
   fly scale count 1
   ```

7. **Check that it's running.** `fly status` should show one machine in the `started` state, and `fly logs` should show no Python errors.

Fly may warn that nothing is listening on a port. You can ignore that, because the bot connects out to Telegram and doesn't serve a website. If it warns that your build context is huge, you're deploying from a crowded folder; move the five files into their own folder.

Step 5: Test it

Open your bot in Telegram and send it a message. A working setup replies with "✓ Saved to Notesnook" within a couple of seconds. The note then appears in Notesnook after its next sync: pull down to refresh on mobile, or click the sync icon on desktop.

Try these in order:

You send	You should get
`Hello world`	A note titled "Hello world"
A message starting with `- [ ] milk`	A checklist note titled like "Telegram note, Oct 15, 8:15 AM"
A photo with the caption `Receipt`	A note titled "Receipt" with the photo embedded, and the reply "✓ Saved to Notesnook (media embedded)"

Then send this as one message:

```
Groceries:
- [ ] milk
- [x] eggs
```

You should get a note titled "Groceries" containing a checklist whose header also reads "Groceries", with milk unchecked and eggs checked.

Two last checks:

- Run `fly machine restart`, wait about 10 seconds, and send another message. The bot should answer without any extra steps.
- Ask a friend to message the bot. It should stay silent.
- Send `/help` and `/ver`. The bot should reply with the usage guide and its version number.

Writing notes

Formatting you apply in Telegram (bold, italic, underline, strikethrough, links, inline code, code blocks, quotes) carries over to the note. Typed markdown works for these line prefixes:

You type	Notesnook gets
`- item`, `* item` or `• item`	Bullet list
`1. item` or `1) item`	Numbered list
`- [ ] item`	Unchecked todo
`- [x] item`	Checked todo
`# Heading` up to `###### Heading`	Heading

Titles. The first line of your message becomes the note title. If the message starts with a list item, the title is generic instead, like "Telegram note, Oct 15, 8:15 AM", so the first item isn't repeated as the title.

Commands. Send `/help` to see a usage guide inside Telegram, including these markdown shortcuts. Send `/ver` to see the bot's version, the Fly.io region it runs in, its time zone and whether media embedding is on.

Checklist headers. Notesnook shows a header row above every checklist. Put a line directly above your todos, with no blank line between, and it becomes that header's title:

```
Groceries:
- [ ] milk
- [ ] eggs
```

With a blank line between, or no line above, the header reads "Untitled".

Photos and files. The Inbox API has no attachment field, so the bot embeds media in the note's HTML, the way the Notesnook email worker does. Photos and image files (JPEG, PNG, GIF, WebP) become embedded images, and text files (.txt, .md, .csv, .json, code and similar) become code blocks. Each file can be up to 5 MB, and a whole note up to 8 MB. Videos, voice notes and other file types get a placeholder line such as "[Video attached in Telegram, not embedded]" plus the caption, if any. Each photo in an album becomes its own note. Embedded images are part of the note, not Notesnook attachments, and Notesnook's documentation suggests it may strip embedded images, but in testing they display correctly. If they ever stop appearing, run `fly secrets set EMBED_MEDIA=0` to switch embedding off, and use your phone's share sheet for real attachments.

Limits. Lists don't nest, and numbering always starts at 1. A message with a code block, a quote, or bold text that runs across several lines skips the list conversion and is saved as simple text, so nothing is lost.

Troubleshooting

Start with `fly logs`. It shows startup errors and every request the bot handles.

What you see	Likely cause	Fix
The bot never replies	Wrong user ID, bot not running, or two machines running	Check your ID with @userinfobot, then run `fly status` and make sure exactly one machine is started
The app crashes at startup with a `KeyError`	A secret is missing	Run `fly secrets list`; all three secrets must appear
Reply says "Failed (401)"	The Inbox key is wrong, expired or revoked	Create a new key, then run `fly secrets set NOTESNOOK_INBOX_KEY="new-key"`
Reply says "Failed (403)"	The Inbox API is turned off	Turn on Enable Inbox API in Notesnook settings
Reply says "Failed (400)"	Notesnook rejected the note; the reply names the field	Check that `bot.py` matches this guide
Reply says "Failed (429)"	Rate limit of 60 requests a minute	Wait a minute and resend
Reply says "Saved" but no note appears	Notesnook hasn't synced yet	Force a sync, then check Failed inbox items under Settings > Account > Inbox
A note arrives as a block of raw markup	The HTML had an unclosed tag	Not expected with this bot; note the exact message you sent
Deploy is slow and warns about a huge build context	Deploying from a crowded folder	Use a dedicated folder with the `.dockerignore` file

What you see	Likely cause	Fix
A photo note arrives without the picture	The photo was over 5 MB, or Notesnook changed how it handles embedded images	Read the bot's reply for "not embedded". If images never show, run fly secrets set EMBED_MEDIA=0

Update the bot. Edit `bot.py` and run `fly deploy` again.

Replace a key. Turning the Inbox API off revokes every key. After turning it back on, create a new key and run `fly secrets set NOTESNOOK_INBOX_KEY="new-key"`. Setting a secret restarts the bot.

Stop or remove it. `fly scale count 0` pauses the bot. `fly apps destroy your-unique-app-name` deletes it.

Limitations and security

- **Attachments are embedded, not attached.** Photos and text files are embedded in the note. Videos, voice notes and other files get a placeholder line.
- **Beta API.** The Notesnook Inbox API may change. If the bot breaks after a Notesnook update, check the official guide.
- **One user per bot.** The bot answers a single Telegram user ID. Anyone who wants their own bridge should deploy their own copy with their own bot token and Inbox key.
- **Protect your secrets.** Never put the bot token or Inbox key in `bot.py`, a Git repository or a screenshot. If either leaks, send `/revoke` to @BotFather for the bot token, and delete the key under Settings > Inbox in Notesnook.
- **Privacy.** Telegram bot chats are not end-to-end encrypted, so Telegram can read your messages. Notesnook's hosted inbox server receives each note over HTTPS, encrypts it with your public key and forwards it; its documentation says it stores nothing. If you'd rather not send notes through it, Notesnook documents self-hosting the inbox server.